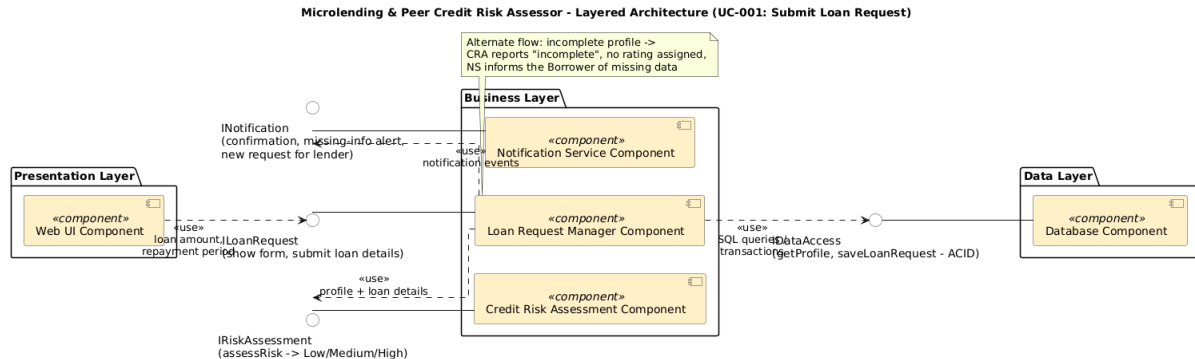


SE - LAB 3

PES1UG24CS401 - Saiesh Shankar



#	Component	Responsibility
1	Web UI Component	Displays the loan form, collects the amount and repayment period, and shows confirmation or errors
2	Loan Request Manager Component	Orchestrates UC-001: validates the loan details, retrieves the profile, assigns the rating, and stores the request
3	Credit Risk Assessment Component	Computes the Low/Medium/High rating from debt-to-income ratio, repayment history, and employment stability (FR002)
4	Notification Service Component	Confirms submission, alerts the Borrower to missing profile data, and makes the request visible to Peer Lenders
5	Database Component	Stores financial profiles, loan requests, and risk ratings with ACID guarantees

We chose **Layered Architecture** for the Microlending & Peer Credit Risk Assessor System.

Why did we Choose layered Architecture?

The system is organised into three layers. The Presentation layer holds the Web UI. The Business layer holds the Loan Request Manager, Credit Risk Assessment, and Notification Service. The Data layer holds the Database. We compared this with Microservices, which would add operational complexity and distributed transactions, and with Client-Server, which gives no internal separation of business logic.

Reason 1: Risk logic is isolated and replaceable.

The credit risk rules (debt-to-income ratio, repayment history, employment stability) are the core of the system (FR002). In a layered design they live in their own component behind the IRiskAssessment interface. The scoring model can change without touching the UI or the database. The Loan Request Manager enforces the rule that a loan is submitted only after a valid rating exists (FR003). This is a single, clear control point for the happy flow and for the incomplete-profile alternate flow.

Reason 2: Financial data needs strong consistency.

NFR001 and NFR002 require ACID transactions, no double-spending, and recoverable records. A single centralised Data layer behind IDataAccess guarantees this naturally. Microservices would spread loan, rating, and repayment data across services and would need distributed transactions or eventual consistency. Both are risky for money.

Security :

Only the Loan Request Manager is allowed to call the Database. The UI cannot reach financial data directly, so every request passes through validation and business rules first. This reduces the attack surface, supports input validation and SQL-injection protection in one place, and makes access control and audit logging on borrower financial profiles simple to enforce.

Performance benefit:

Components are called in-process, and the whole submission (validate, fetch profile, compute rating, store) runs as one database transaction. This avoids the network latency of microservice calls. It keeps loan submission responsive, and the risk calculation is a lightweight rule-based computation. The layers can also be scaled together as the number of borrowers and lenders grows.